Week 8: Introduction to Cloud Computing

Python 200 | Mentor Session

Welcome to the Cloud

Over the past seven weeks you built a strong foundation:
statistical thinking, classical ML, deep learning, LLMs, RAG, and agents.

For the **final four weeks**, we move that work online — to the cloud.

Cloud platforms are the infrastructure that makes modern data engineering possible. You'll be using **Microsoft Azure**.

What Cloud Computing Actually Is

Running your code and storing your data on someone else's computers — computers that you can rent by the minute, scale up or down on demand, and access from anywhere.

Before cloud: you bought servers, racked them, maintained them, and hoped you sized them right.

After cloud: you provision what you need, pay for what you use, and let the provider handle the hardware.

> **Discussion:** What are the advantages of renting compute on demand vs. owning your own servers? Are there situations where owning is still better?

Three Ways to Use the Cloud

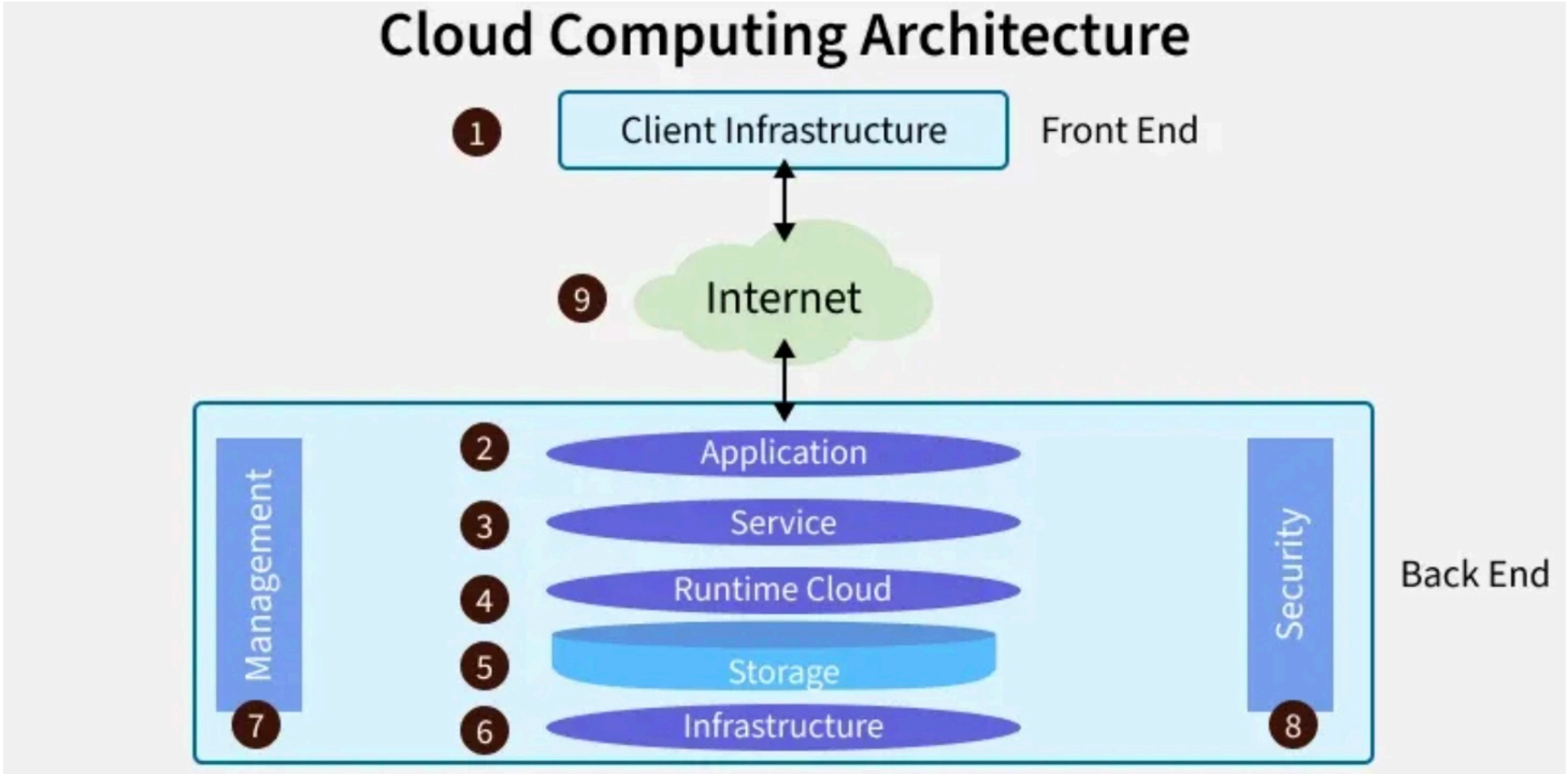
Model	You manage	Provider manages	Example
IaaS	OS, runtime, app	Physical hardware	Azure VMs
PaaS	Your app and data	OS, runtime, infra	Azure App Service
SaaS	Nothing	Everything	Gmail, Salesforce

Most data engineering work lives in IaaS and PaaS.

Core Cloud Services You'll Use

- **Compute** — virtual machines, containers, serverless functions
- **Storage** — object storage (files), databases, data lakes
- **Networking** — virtual networks, load balancers, DNS
- **Managed services** — hosted Postgres, hosted ML APIs, hosted Kafka

The cloud's value: managed services mean you don't maintain the infrastructure, just use it.


Cloud Computing Architecture
1
Client Infrastructure
Front End
9
Internet
2
Application
3
Service
4
Runtime Cloud
5
Storage
6
Infrastructure
Management
7
Security
8
Back End

The Three Major Providers

Provider	Known for
AWS (Amazon)	Largest market share, widest service catalog
Azure (Microsoft)	Strong in enterprise and nonprofit; deep Microsoft ecosystem integration
GCP (Google)	Strong ML/AI tooling; BigQuery

The concepts transfer across providers. The commands and UI differ.

Why Azure for This Course

- Strong presence in nonprofits and government — relevant for Code the Dream's mission
- Deep integration with tools many organizations already use (Microsoft 365, Active Directory)
- Generous free tier for learners
- The patterns you learn here apply directly to AWS and GCP

The Azure Portal

The Portal is Azure's web-based control panel — portal.azure.com

From here you can:

- Create and manage resources (storage accounts, VMs, databases)
- View costs and usage
- Monitor running services
- Set up access control

It's useful for exploration and one-off tasks. For repeatable work, you'll use the CLI.

The Azure CLI

A command-line tool for managing Azure resources — good for automation and scripts.

```
# Log in
az login

# Show your current subscription
az account show

# List resource groups
az group list --output table
```

Install: `winget install Microsoft.AzureCLI` (Windows) or `brew install azure-cli` (Mac)

> **Discussion:** Why would you want a CLI for cloud management rather than just using the web portal?

Azure Cloud Shell

A terminal that runs **inside your browser**, pre-authenticated to your Azure account.

- No install required
- Always has the Azure CLI, Python, and common tools
- Useful for quick tasks when you're not at your own machine

Access via the `>_` icon in the top bar of the Azure Portal.

Demo: Navigating the Portal *(Lesson 2)*

Open: Azure Portal → portal.azure.com

Walk through together:

1. Find your **subscription** and **resource group**
2. Open **Cloud Shell** from the top bar
3. Run `az account show` to confirm you're in the right account
4. Explore the left-hand menu: Storage, Virtual Machines, etc.

Goal for today: everyone logged in, Cloud Shell working, and comfortable finding their way around.

> **Discussion:** What does a "resource group" mean? Why group resources together?

SSH Keys: What and Why

When you connect to a remote server, you need to prove who you are.

Password login — you type a password. Convenient but weaker.

SSH key pair — you generate two linked keys:

- **Private key** — stays on your machine, never shared
- **Public key** — uploaded to the server

The server knows your public key. To log in, your machine proves it holds the matching private key — without sending it anywhere.

> **Discussion:** Why is proving you have a key mathematically safer than sending a password over the network?

Demo: SSH Setup *(Lesson 2)*

In Cloud Shell or your terminal:

```
# Generate a key pair
ssh-keygen -t ed25519 -C "your_email@example.com"

# Print your public key (this is what you share)
cat ~/.ssh/id_ed25519.pub
```

Students: paste your public key into the course setup form, or upload it to your Azure VM configuration.

Week 8 Wrap-Up

- **Cloud computing** — on-demand compute and storage; rent what you need, pay for what you use
- **IaaS / PaaS / SaaS** — different levels of abstraction; most data engineering uses IaaS and PaaS
- **Core services** — compute, storage, networking, managed services
- **Azure Portal** — web UI for exploration and one-off tasks
- **Azure CLI** — command-line tool for repeatable, scriptable work
- **SSH keys** — the standard way to authenticate to remote servers

Takeaway: Cloud fundamentals transfer across providers. What you learn in Azure applies directly to AWS and GCP — the commands differ, the concepts don't.

> **Discussion:** What part of the cloud setup felt most unfamiliar? What clicked?